Core Evidence 3: Decision Log

IT3091 Machine Learning — Industry Explorer Track

Group ID: `2026-AI-36`

Project: ChefCast AI — Restaurant Revenue & Product Demand Forecasting

Version: Final Submission — October 2026

> *The Decision Log is the audit trail of every critical methodological choice made during this project. Under the IT3091 assessment rubric, marks are awarded for **defensible reasoning** — every decision below is backed by domain logic, statistical evidence, or published ML theory rather than arbitrary preference.*

Formal Decision Table

Decision ID	Stage	Decision Area	Options Considered	Choice Made	Evidence & Technical Justification
DEC-01	Project Setup	Track Selection	(1) Guided Data Track — curated academic dataset (2) Industry Explorer Track — authentic POS data	**Industry Explorer Track**	127,093 authentic POS transactions capture real hospitality dynamics: transaction voids, price anomalies, zero-demand Poya days, and raw data quality challenges absent from curated datasets. Industry data maximises assessed learning value and earns eligibility for 10 bonus marks.
DEC-02	Problem Framing	Lens Architecture	(1) Single Lens — revenue forecasting only (2) Disconnected dual lenses (no coherent linkage) (3) Coherent Primary + Secondary Lens	**Coherent Dual-Lens: Revenue Planning (Lens 1) + Product Demand Planning (Lens 2)**	Revenue forecasts alone do not guide kitchen preparation — chefs cook portions, not rupees. The secondary lens directly operationalizes the primary lens: financial targets → physical kitchen batch quantities. This creates a closed, coherent analytical loop that satisfies the business decision all the way to morning cooking.

DEC-03	Problem Framing	Product Scope	(1) Model all 274 catalog items (2) Ad-hoc manual selection (3) Systematic 5-criterion framework	**11 core fresh handmade items via 5-criterion framework**	Items selected by: (i) stakeholder-confirmed kitchen preparation, (ii) ≥30% revenue share, (iii) high sales frequency, (iv) perishable shelf life < 48h, (v) continuous 2024–2026 data coverage. Packaged beverages and cigarettes have zero kitchen preparation burden and no spoilage risk — including them dilutes the forecasting signal and adds irrelevant noise. Result: 11 items generating ~47.0% of cleaned store revenue.
DEC-04	Data Cleaning	Anomaly Handling	(1) Ignore anomalies — use raw data (2) Absolute-value transformation on negatives (3) Explicit categorical filtering with audit logging	**Explicit Filtering & Audit Logging**	Negative quantities (931 rows) are voided transactions, not negative demand. Absolute-value conversion would falsely add demand that never existed. Negative prices (72 rows) represent promotional discounts — including them artificially depresses revenue calculations. Extreme prices (***2.41 × 10^{11}*** LKR; 52 rows) are keying errors — no dish costs more than a small car. All filters are domain-justified and logged.
DEC-05	Preprocessing	Inactive Calendar Dates	(1) Drop zero-sales dates entirely (2) Linear temporal interpolation (3) Reindex full calendar with zero-filling	**Reindex full calendar; insert 0.0 for zero-sales dates**	Dropping dates breaks the continuity of weekly lag features (***{lag}_7***, ***{lag}_{14}***). Linear interpolation invents demand that was never recorded and may introduce spurious price-level hallucinations. Zero-filling preserves temporal structure and is correctly characterized as *"no qualifying POS sales recorded"* — acknowledging the ambiguity between physical closure and POS terminal downtime without fabricating data.

DEC-06	Feature Engineering	Data Leakage Prevention	(1) Unshifted rolling averages (naive approach) (2) Strict shift=1 on all lag and rolling features	**Strict shift=1 on all past aggregations**	In production, day ***t***'s sales figures are unknown at 06:00 AM when the kitchen prepares. Computing `roll_mean_7(t)` without shift uses day ***t***'s actual revenue in the predictor for day ***t*** — a textbook lookahead leak. The shift=1 ensures ***{roll_mean_7}(t) = \frac{1}{7}Σ_{k=1}^{7} y_{t-k}***, using only information observable before day ***t*** begins.
DEC-07	Evaluation	Data Splitting Strategy	(1) Random K-Fold cross-validation (2) Test-set champion selection (prohibited) (3) Chronological Walk-Forward 70/15/15 split with validation-only selection	**Chronological Walk-Forward Split; champion selected strictly on Validation WAPE**	Random K-Fold violates temporal ordering — future sales can appear in training folds, making validation scores unrealistically optimistic. Selecting champions on the test set creates selection bias (data dredging). The Chronological 70/15/15 split mirrors real deployment: models trained on historical data must generalize to genuinely unseen future dates. Test set was evaluated exactly once per lens.
DEC-08	Evaluation	Primary Metric	(1) MAPE — Mean Absolute Percentage Error (2) WAPE + MAE + RMSE + R^2	**WAPE as primary selection metric; MAE, RMSE, R^2 as secondary diagnostics**	MAPE suffers from division-by-zero when daily demand is zero (e.g., WATALAPPAN selling 0 on Poya days → MAPE = ∞). MAPE also inflates to meaningless percentages for low-volume items selling 1–2 units daily. WAPE aggregates absolute errors relative to total volume, remaining stable and interpretable regardless of zero-demand dates — the gold standard metric in retail supply-chain ML (Walmart, Amazon Demand Forecasting).

DEC-09	Modeling	Candidate Model Lineup	(1) Single algorithm only (2) Only linear models (3) Diverse portfolio: Baseline + Linear + Bagging + Boosting + Stacking	**Seasonal Naive Baseline + Ridge + Random Forest + LightGBM + XGBoost + Stacking Ensemble**	Diversity across model families is essential for defensible comparison. A single algorithm cannot distinguish whether performance gains are due to ML generally or the specific algorithm. The portfolio spans: simple heuristic (Naive), regularized linear (Ridge), bagging (RF), two complementary gradient boosters (LightGBM, XGBoost), and stacking — ensuring all major ML paradigms are tested.
DEC-10	Modeling	Stacking Cross-Validation	(1) Random CV (n=3) (2) Walk-Forward TimeSeriesSplit (n=3)	**Custom TimeSeriesStackingRegressor with TimeSeriesSplit(n_splits=3)**	Standard `sklearn.StackingRegressor.cross_val_predict` uses random CV, generating meta-features contaminated by future training observations. Our custom `TimeSeriesStackingRegressor` uses `TimeSeriesSplit(n_splits=3)` to produce chronologically safe out-of-fold predictions — ensuring the meta-learner is trained on structurally uncontaminated, temporally honest predictions.
DEC-11	Deployment	Champion Retraining for Production	(1) Deploy on training-set-only trained model (2) Retrain champion on Train + Validation combined	**Retrain champion on Train + Validation combined for deployment**	The validation set serves its purpose (champion selection) and should be incorporated into the production training window to maximize training data volume. The test set is not used for retraining — it serves only as the final unbiased benchmark. The forward forecasting engine uses this extended-window champion.

DEC-12	Deployment	User Interface Approach	(1) Static Jupyter Notebook report only (2) Simple test-set replay table (3) Interactive Streamlit UI with recursive out-of-sample forecasting	**Interactive Streamlit Decision Dashboard with genuine out-of-sample recursive forecasting**	A static notebook cannot support real-time operational use. The dashboard must generate forward projections beyond historical data, support buffer scenario adjustment, and allow kitchen staff to use it on a daily basis. Recursive autoregressive forecasting (`src/forecasting.py`) enables true future projection from the last available POS date (September 7, 2026 → September 8–14 forecasts).
DEC-13	Responsible AI	Prediction Safety Ceiling	(1) No ceiling — report model output raw (2) Cap at rolling 30-day maximum × 1.3	**Hard ceiling: max prediction = 1.3 × rolling 30-day maximum demand**	Gradient boosting and recursive error accumulation can occasionally generate implausibly large predictions on unusual future dates. A hard safety ceiling prevents the dashboard from displaying physically impossible kitchen prep quantities (e.g., 500 portions of WATALAPPAN from a dessert station that has never exceeded 25 per day).

Decision Dependency Map

```
DEC-01 (Industry Track) ■■→ DEC-02 (Dual Lens Architecture) ■■→ DEC-03 (Product Scope) ■ ■■→ DEC-04 (Anomaly Handling) ■ ■■→ DEC-05 (Calendar Reindexing) ■ ■■→ DEC-06 (Leakage Prevention) ■ ■■→ DEC-09 (Model Lineup) ■ ■■→ DEC-10 (Stacking CV) ■ ■■→ DEC-07 (Split Strategy) ■ ■■→ DEC-08 (Metric Choice) ■ ■■→ Lens 1 Champion: LightGBM ■ ■■→ Lens 2 Champion: Ridge ■■→ DEC-11 (Production Retraining) ■■→ DEC-12 (Streamlit UI) ■■→ DEC-13 (Safety Ceiling)
```

Key Learning: What Would Have Gone Wrong Without These Decisions?

If We Had NOT Made This Decision	Consequence
Skipped DEC-06 (no shift=1)	Models would appear to achieve near-0% WAPE on validation but fail completely in production — a classic academic warning about lookahead leakage.
Used random K-Fold (not DEC-07)	Validation WAPE would be ~10–15% (falsely optimistic) due to future data appearing in training folds. Test set performance would be 30–45% worse than validation, making champion selection meaningless.
Used MAPE not WAPE (DEC-08)	Aggregate metric would be dominated by infinite values from zero-demand Poya days, making all model comparisons statistically meaningless.
Included all 274 products (not DEC-03)	Zero-inflation across hundreds of sporadic items would collapse model performance; kitchen staff would receive prep targets for packaged cigarettes and bottled water — nonsensical for fresh food planning.